

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO



FACULTAD DE INGENIERÍA

INGENIERÍA EN COMPUTACIÓN

Sistemas Distribuidos

Mexi Banco: de un monolito a servicios que se encuentran solos· sábado 10 de octubre (S7)

NOMBRE DE LA ALUMNA: Muñoz San Agustin Victoria Monserrat

NÚMERO DE CUENTA: 320094187

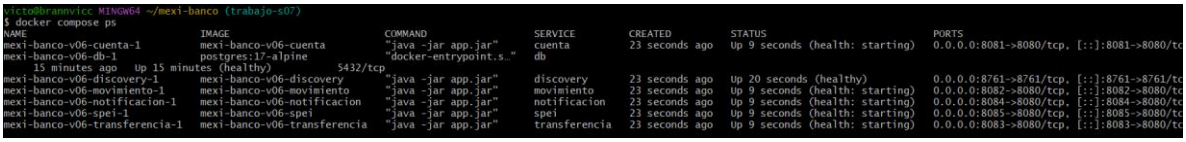
NOMBRE DEL PROFESOR: Oscar Manuel Ruiz Hurtado

GRUPO: 02

SEMESTRE 2027-1

FECHA DE ENTREGA: sábado 10 de octubre de 2026

PASO 1: La historia (22:45 MX)



¿Qué se rompió al pasar de llamada a método a HTTP?

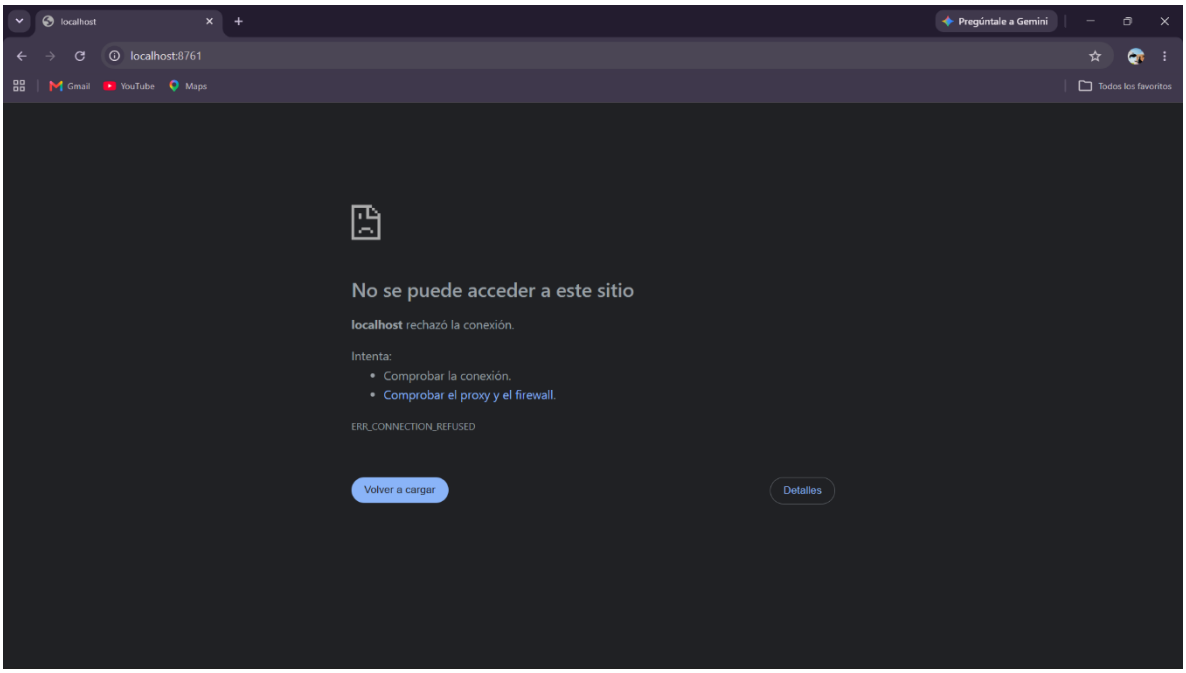
En la arquitectura monolítica (S05), todo el sistema se ejecutaba dentro de un mismo proceso de Java. Una transferencia se realizaba llamando a métodos locales dentro de una única transacción. Si ocurría un fallo al abonar el dinero a la cuenta destino, la base de datos realizaba un rollback de forma automática de toda la operación, garantizando el principio de "todo o nada".

Al separar la aplicación en 5 microservicios independientes con HTTP (S06):

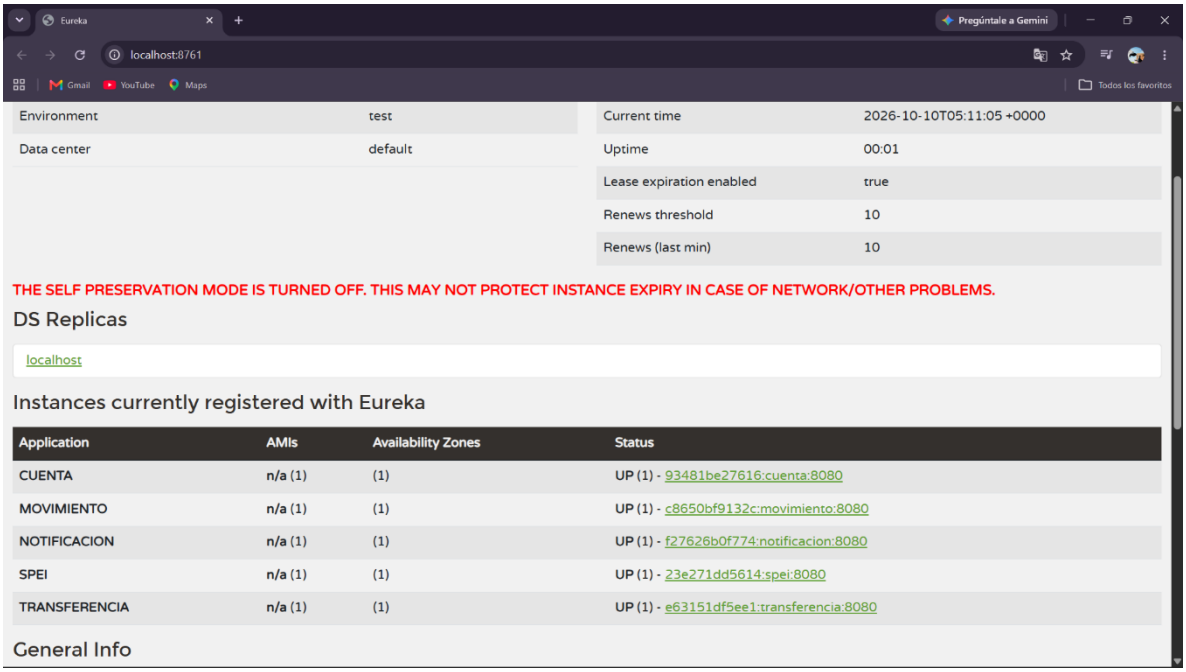
- Pérdida del @Transactional global: Cada microservicio tiene su propia base de datos y sus propias transacciones locales independientes.
- Inconsistencia de datos: Cuando transferencia solicita un cargo a cuenta por HTTP, ese cargo queda confirmado en la base de datos de origen al instante. Si la petición HTTP posterior para realizar el abono en la cuenta destino falla (por red o error del servicio), el cargo inicial ya no se deshace automáticamente.
- Manejo de red y direcciones: Aparecieron problemas de localización y falta de tolerancia a fallos ante caídas de red o de instancias.

PASO 2: Directorio (23:10 MX)

Antes: Al intentar abrir http://localhost:8761, el navegador no cargaba la página (ERR_CONNECTION_REFUSED) porque el servidor de descubrimiento aún no estaba activo ni publicado.



Después: Al habilitar @EnableEurekaServer y levantar el contenedor discovery, la página http://localhost:8761 mostró el panel de Eureka con los 5 servicios registrados en estado UP (1): CUENTA, MOVIMIENTO, NOTIFICACION, SPEI y TRANSFERENCIA.



Consulta XML del servicio CUENTA (http://localhost:8761/eureka/apps/CUENTA):

This XML file does not appear to have any style information associated with it. The document tree is shown below.

<application>
<name>CUENTA</name>
<instance>
<instanceId>21ec7e7c72e6:cuenta:8080</instanceId>
<hostName>172.18.0.4</hostName>
<app>CUENTA</app>
<ipAddr>172.18.0.4</ipAddr>
<status>UP</status>
<overriddenstatus>UNKNOWN</overriddenstatus>
<port enabled="true">8080</port>
<securePort enabled="false">443</securePort>
<countryId>1</countryId>
<dataCenterInfo class="com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo">
<name>MyOwn</name>
</dataCenterInfo>
<leaseInfo>
<renewalIntervalInSecs>5</renewalIntervalInSecs>
<durationInSecs>15</durationInSecs>
<registrationTimestamp>1791609801992</registrationTimestamp>
<lastRenewalTimestamp>1791611818720</lastRenewalTimestamp>
<evictionTimestamp>0</evictionTimestamp>
<serviceUpTimestamp>1791609801564</serviceUpTimestamp>
</leaseInfo>
<metadata>
<management.port>8080</management.port>
</metadata>
<homePageUrl>http://172.18.0.4:8080/</homePageUrl>
<statusPageUrl>http://172.18.0.4:8080/actuator/info</statusPageUrl>
<healthCheckUrl>http://172.18.0.4:8080/actuator/health</healthCheckUrl>

<vipAddress>cuenta</vipAddress>
<secureVipAddress>cuenta</secureVipAddress>
<isCoordinatingDiscoveryServer>false</isCoordinatingDiscoveryServer>
<lastUpdatedTimestamp>1791609801992</lastUpdatedTimestamp>
<lastDirtyTimestamp>1791609801500</lastDirtyTimestamp>
<actionType>ADDED</actionType>
</instance>
<instance>
<instanceId>92a4974ea0de:cuenta:8080</instanceId>
<hostName>172.18.0.11</hostName>
<app>CUENTA</app>
<ipAddr>172.18.0.11</ipAddr>
<status>UP</status>
<overriddenstatus>UNKNOWN</overriddenstatus>
<port enabled="true">8080</port>
<securePort enabled="false">443</securePort>
<countryId>1</countryId>
<dataCenterInfo class="com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo">
<name>MyOwn</name>
</dataCenterInfo>
<leaseInfo>
<renewalIntervalInSecs>5</renewalIntervalInSecs>
<durationInSecs>15</durationInSecs>
<registrationTimestamp>1791609646585</registrationTimestamp>
<lastRenewalTimestamp>1791611817267</lastRenewalTimestamp>
<evictionTimestamp>0</evictionTimestamp>
<serviceUpTimestamp>1791609646071</serviceUpTimestamp>
</leaseInfo>
<metadata>
<management.port>8080</management.port>
</metadata>

<homePageUrl>http://172.18.0.11:8080/</homePageUrl>
<statusPageUrl>http://172.18.0.11:8080/actuator/info</statusPageUrl>
<healthCheckUrl>http://172.18.0.11:8080/actuator/health</healthCheckUrl>
<vipAddress>cuenta</vipAddress>
<secureVipAddress>cuenta</secureVipAddress>
<isCoordinatingDiscoveryServer>false</isCoordinatingDiscoveryServer>
<lastUpdatedTimestamp>1791609646585</lastUpdatedTimestamp>
<lastDirtyTimestamp>1791609645262</lastDirtyTimestamp>
<actionType>ADDED</actionType>
</instance>
<instance>
<instanceId>96eb788c30ff:cuenta:8080</instanceId>
<hostName>172.18.0.10</hostName>
<app>CUENTA</app>
<ipAddr>172.18.0.10</ipAddr>
<status>UP</status>
<overriddenstatus>UNKNOWN</overriddenstatus>
<port enabled="true">8080</port>
<securePort enabled="false">443</securePort>
<countryId>1</countryId>
<dataCenterInfo class="com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo">
<name>MyOwn</name>
</dataCenterInfo>
<leaseInfo>
<renewalIntervalInSecs>5</renewalIntervalInSecs>
<durationInSecs>15</durationInSecs>
<registrationTimestamp>1791609649142</registrationTimestamp>
<lastRenewalTimestamp>1791611815655</lastRenewalTimestamp>
<evictionTimestamp>0</evictionTimestamp>
<serviceUpTimestamp>1791609648628</serviceUpTimestamp>
</leaseInfo>

<metadata>
<management.port>8080</management.port>
</metadata>
<homePageUrl>http://172.18.0.10:8080/</homePageUrl>
<statusPageUrl>http://172.18.0.10:8080/actuator/info</statusPageUrl>
<healthCheckUrl>http://172.18.0.10:8080/actuator/health</healthCheckUrl>
<vipAddress>cuenta</vipAddress>
<secureVipAddress>cuenta</secureVipAddress>
<isCoordinatingDiscoveryServer>false</isCoordinatingDiscoveryServer>
<lastUpdatedTimestamp>1791609649142</lastUpdatedTimestamp>
<lastDirtyTimestamp>1791609648298</lastDirtyTimestamp>
<actionType>ADDED</actionType>
</instance>
</application>

En el compose, transferencia sigue llamando a CUENTA_URL: http://cuenta:8080. ¿Alguien está usando el directorio para encontrar a cuenta?

En este punto, nadie está usando el directorio aún para comunicarse con cuenta. La resolución de http://cuenta:8080 la realiza directamente el servidor DNS interno de Docker Compose, no Eureka. Eureka solo está registrando la información, pero los servicios todavía no consultan al directorio.

PASO 3: Gateway (23:16 MX)

Resultados de las pruebas con curl:

Llamada directa interna a movimiento (curl -i -X POST http://localhost:8082/movimientos):

```
victo@brannvicc MINGW64 ~/mexi-banco (trabajo-s07)
$ curl -i -X POST http://localhost:8082/movimientos
HTTP/1.1 400
Content-Type: application/problem+json
Transfer-Encoding: chunked
Date: Sat, 10 Oct 2026 05:15:07 GMT
Connection: close

{"type":"about:blank","title":"Bad Request","status":400,"detail":"Failed to read request","instance":"
/movimientos"}
```

Explicación: La llamada llega directamente al microservicio de movimiento sin pasar por ningún filtro. Nos responde 400 porque faltan los datos del cuerpo en el POST, demostrando que el endpoint interno está desprotegido en su puerto directo.

Acceso por el Gateway (http://localhost:8080):

- Prueba A (curl -i http://localhost:8080/api/cuentas/000):

```
victo@brannvicc MINGW64 ~/mexi-banco (trabajo-s07)
$ curl -i http://localhost:8080/api/cuentas/000
HTTP/1.1 404 Not Found
transfer-encoding: chunked
X-Instancia: 93481be27616
Content-Type: application/problem+json
Date: Sat, 10 Oct 2026 05:16:04 GMT

{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 000","instance":"/cuent
as/000"}
```

El microservicio cuenta. El Gateway enrutó correctamente la petición hasta cuenta, y fue el microservicio de negocio el que contestó "No existe la CLABE 000".

- Prueba B (curl -i -X POST http://localhost:8080/api/movimientos):

```
victo@brannvicc MINGW64 ~/mexi-banco (trabajo-s07)
$ curl -i -X POST http://localhost:8080/api/movimientos
HTTP/1.1 404 Not Found
Content-Type: application/json
Content-Length: 129
```

El API Gateway. Bloqueó la petición en la entrada porque en el archivo application.yml configuramos la ruta de /api/movimientos para aceptar únicamente el método GET.

Alta y consulta de cuenta a través del Gateway:

- Alta (201 Created): Se ejecutó curl -X POST http://localhost:8080/api/cuentas con la CLABE única S071791609385, devolviendo un estado 201 Created y JSON con saldo inicial de 0.

```
victo@brannvicc MINGW64 ~/mexi-banco (trabajo-s07)
$ LAB_CLABE="S07$(date +%s)"
curl -i -X POST http://localhost:8080/api/cuentas \
  -H 'Content-Type: application/json' \
  -d "{\"clabe\":\"$LAB_CLABE\",\"titular\":\"S07\",\"saldoInicial\":0}"
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 93481be27616
Content-Type: application/json
Date: Sat, 10 Oct 2026 05:16:25 GMT
```

- Consulta (200 OK): Se ejecutó curl http://localhost:8080/api/cuentas/S071791609385, devolviendo 200 OK y devolviendo la información en formato JSON a través del puerto único 8080.

```
victo@brannvicc MINGW64 ~/mexi-banco (trabajo-s07)
$ URL="http://localhost:8080/api/cuentas/$LAB_CLABE"
curl -i "$URL"
echo "$URL"
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 93481be27616
Content-Type: application/json
Date: Sat, 10 Oct 2026 05:16:34 GMT

{"id":1,"clabe":"S071791609385","titular":"S07","saldo":0.00}http://localhost:8080/api/cuentas/S071791609385
```

Paso 4 y 5: Copias y caída (23:20 MX)

Al configurar deploy: replicas: 3 en cuenta dentro del docker-compose.yml y remover la publicación directa de sus puertos de la laptop:

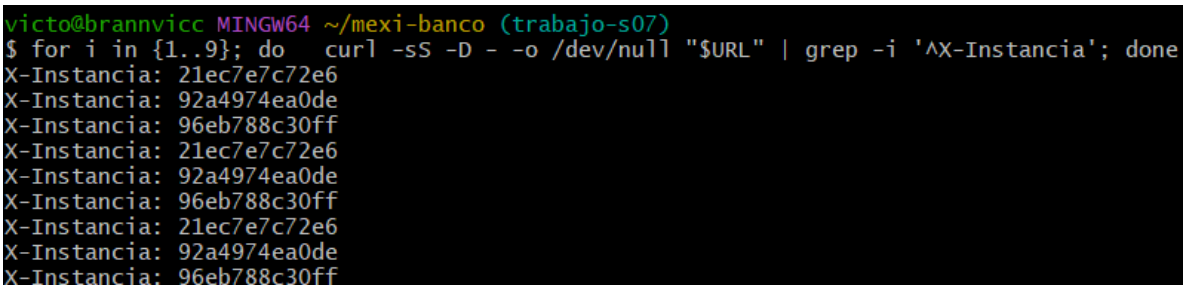
```
victo@brannvicc MINGW64 ~/mexi-banco (trabajo-s07)
$ docker ps --filter name=cuenta --format '{{.ID}}  {{.Names}}  {{.Ports}}'
21ec7e7c72e6  mexi-banco-v06-cuenta-1  8080/tcp
96eb788c30ff  mexi-banco-v06-cuenta-2  8080/tcp
92a4974ea0de  mexi-banco-v06-cuenta-3  8080/tcp
```

En http://localhost:8761, el servicio CUENTA cambió a UP (3).

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
CUENTA	n/a (3)	(3)	UP (3) - 21ec7e7c72e6:cuenta:8080 , 92a4974ea0de:cuenta:8080 , 96eb788c30ff:cuenta:8080
GATEWAY	n/a (1)	(1)	UP (1) - dbc9e1ca7bba:gateway:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - 70d829829aab:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 7536f2214355:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - e3fa83bb32ab:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - 826cbf6826f7:transferencia:8080

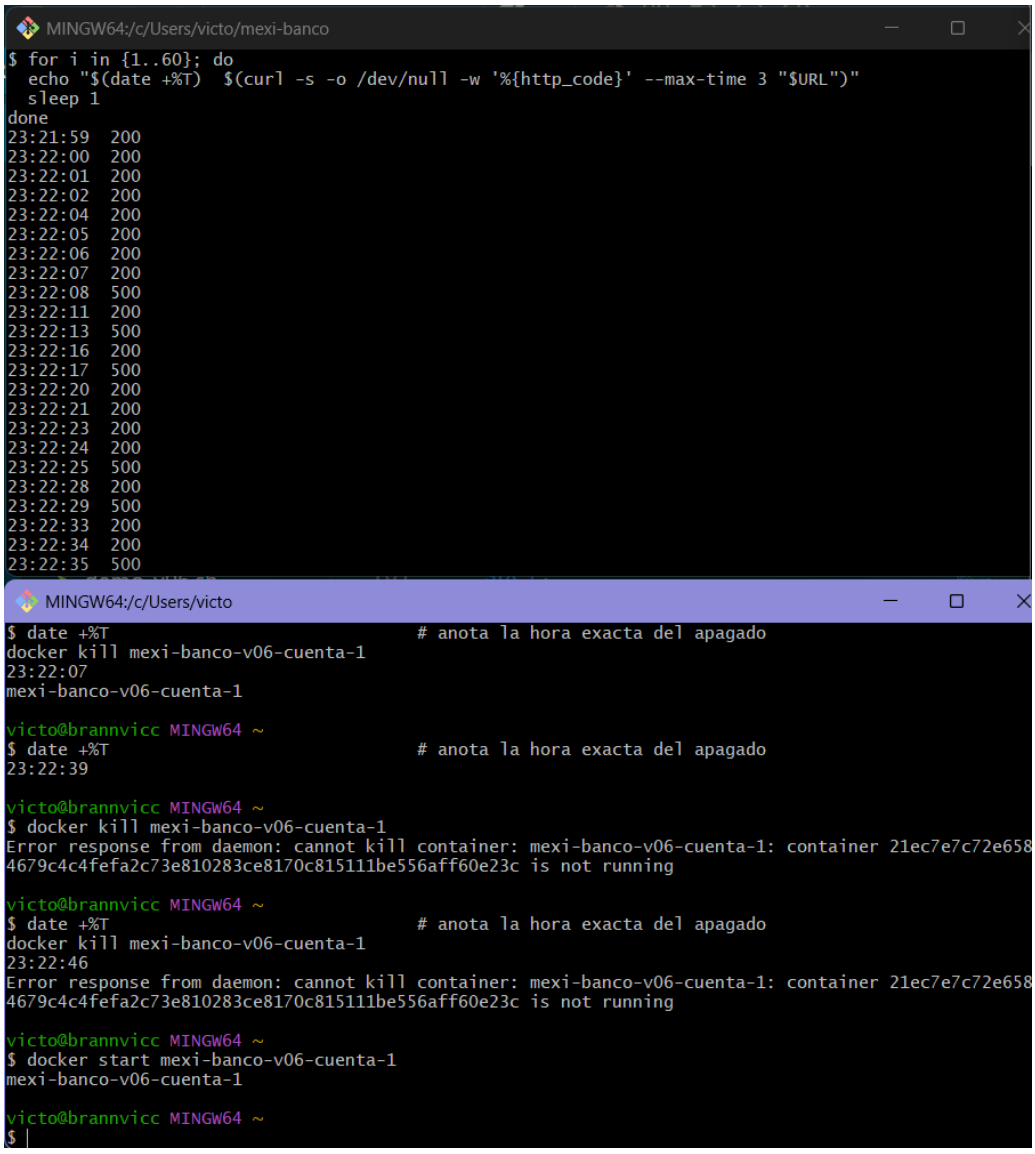
Al ejecutar las 9 consultas consecutivas en la terminal:

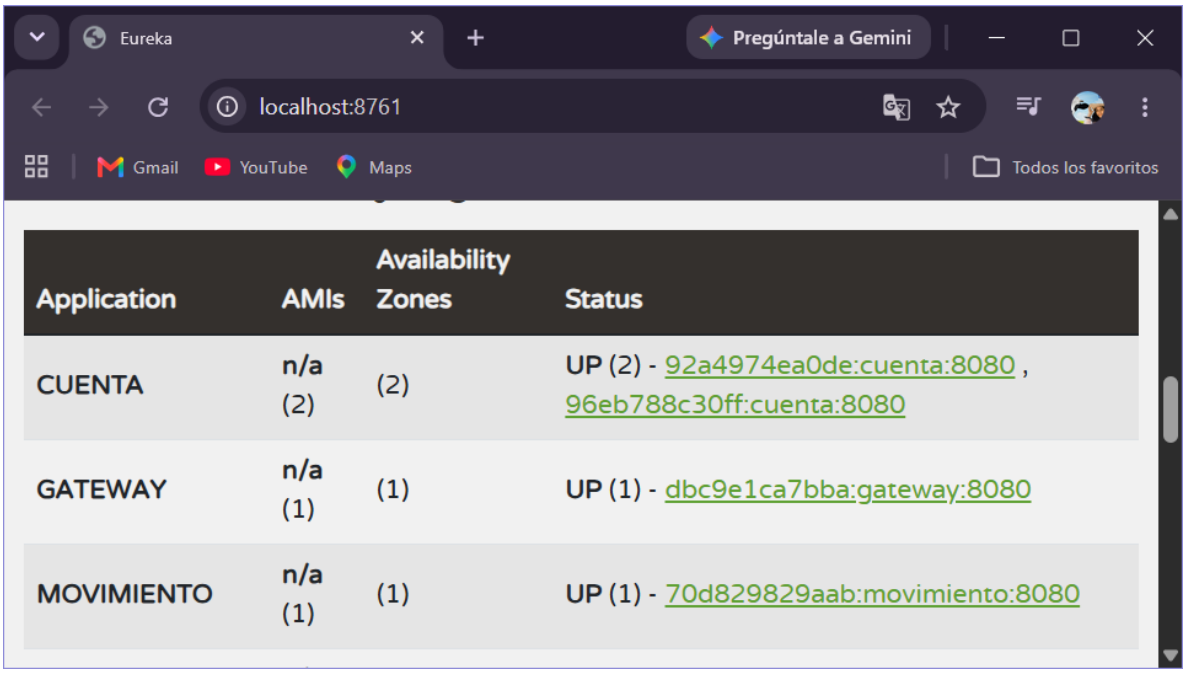


Se observó que los valores de X-Instancia alternaron ordenadamente entre los IDs de las tres réplicas (cuenta-1, cuenta-2 y cuenta-3), confirmando el algoritmo Round Robin de Spring Cloud Gateway.

Simulación de caída con docker kill:

Predicción previa: Las consultas enviadas a la réplica muerta fallarán con código 500/503 temporalmente, mientras que las enviadas a las copias vivas responderán 200. Los errores desaparecerán cuando Eureka remueva la instancia caída.


MINGW64:/c/Users/victo/mexi-banco
$ for i in {1..60}; do
echo "$(date +%T) $(curl -s -o /dev/null -w '%{http_code}' --max-time 3 "$URL")"
sleep 1
done
23:21:59 200
23:22:00 200
23:22:01 200
23:22:02 200
23:22:04 200
23:22:05 200
23:22:06 200
23:22:07 200
23:22:08 500
23:22:11 200
23:22:13 500
23:22:16 200
23:22:17 500
23:22:20 200
23:22:21 200
23:22:23 200
23:22:24 200
23:22:25 500
23:22:28 200
23:22:29 500
23:22:33 200
23:22:34 200
23:22:35 500
MINGW64:/c/Users/victo
$ date +%T # anota la hora exacta del apagado
docker kill mexi-banco-v06-cuenta-1
23:22:07
mexi-banco-v06-cuenta-1
victo@brannvicc MINGW64 ~
$ date +%T # anota la hora exacta del apagado
23:22:39
victo@brannvicc MINGW64 ~
$ docker kill mexi-banco-v06-cuenta-1
Error response from daemon: cannot kill container: mexi-banco-v06-cuenta-1: container 21ec7e7c72e658
4679c4c4fefa2c73e810283ce8170c815111be556aff60e23c is not running
victo@brannvicc MINGW64 ~
$ date +%T # anota la hora exacta del apagado
docker kill mexi-banco-v06-cuenta-1
23:22:46
Error response from daemon: cannot kill container: mexi-banco-v06-cuenta-1: container 21ec7e7c72e658
4679c4c4fefa2c73e810283ce8170c815111be556aff60e23c is not running
victo@brannvicc MINGW64 ~
$ docker start mexi-banco-v06-cuenta-1
mexi-banco-v06-cuenta-1
victo@brannvicc MINGW64 ~
$



- Hora del apagado de la instancia (docker kill mexi-banco-v06-cuenta-1): 23:22:07
- Hora de salida del directorio (paso a UP (2) en tablero): 23:22:39 (~32 segundos después).
- Cantidad de errores observados: Se observaron 7 errores 500 intercalados en las consultas de un total de 45 peticiones.

¿Por qué ocurre esto? Porque el Gateway consulta la lista de instancias a Eureka de forma periódica mediante una caché interna. Durante la ventana de tiempo (~30 s) previa a la expiración del plazo de expiración y refresco de caché, el Gateway continúa enviando 1 de cada 3 peticiones a la réplica muerta.

Se recupera con:

```
victo@brannvicc MINGW64 ~/mexi-banco (trabajo-s07)
$ docker start mexi-banco-v06-cuenta-1
mexi-banco-v06-cuenta-1
```

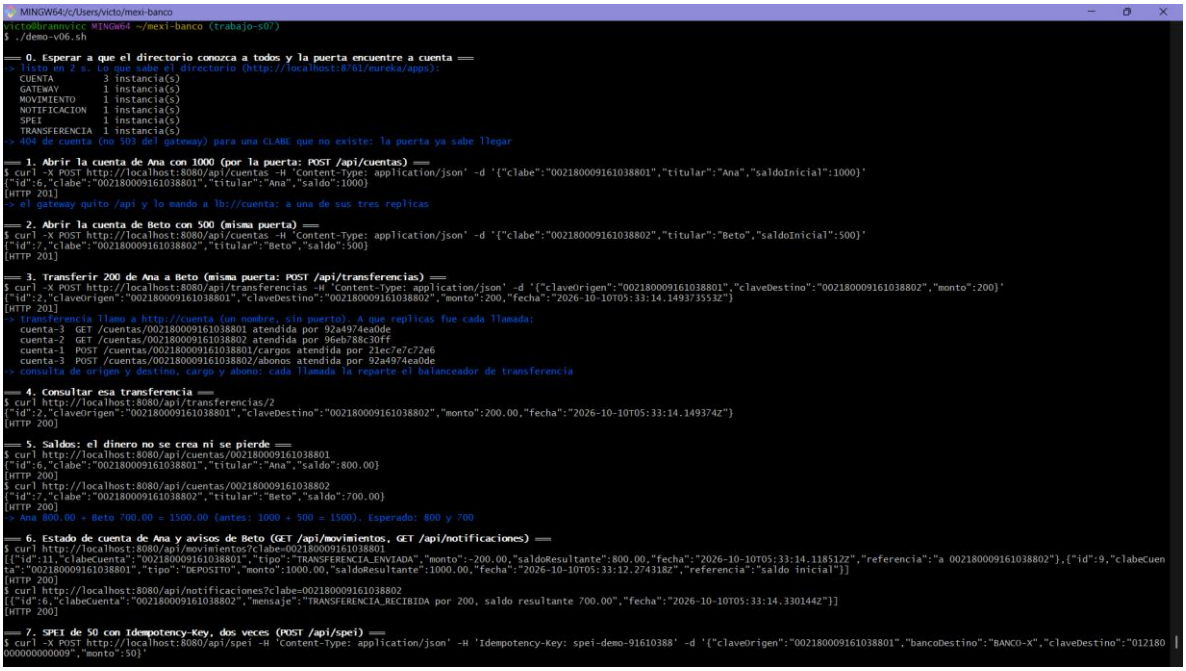
Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
CUENTA	n/a (3)	(3)	UP (3) - 21ec7e7c72e6:cuenta:8080 , 92a4974ea0de:cuenta:8080 , 96eb788c30ff:cuenta:8080
GATEWAY	n/a (1)	(1)	UP (1) - dbc9e1ca7bba:gateway:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - 70d829829aab:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 7536f2214355:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - e3fa83bb32ab:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - 826cbf6826f7:transferencia:8080

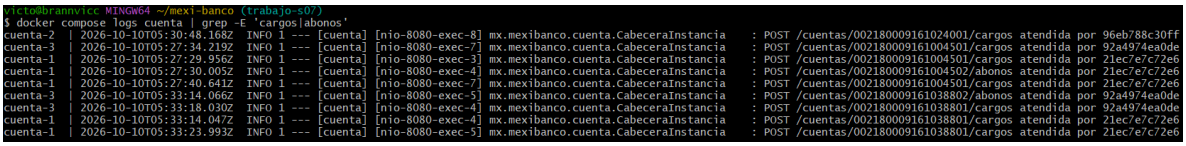
Cierre

1. El directorio resuelve la localización dinámica de servicios en la red, permitiendo que las aplicaciones se encuentren por nombre sin tener que escribir direcciones IP o puertos a mano.
2. El gateway resuelve disponer de una puerta de entrada única e unificada hacia el exterior, aplicando filtros, ocultando puertos internos y gestionando reglas de acceso HTTP.
3. El balanceador resuelve la distribución equilibrada del tráfico de peticiones entre múltiples réplicas disponibles y la tolerancia a fallos ante caídas.
4. Cuando murió una copia vi errores de HTTP 500/503 intercalados durante aproximadamente 30 segundos porque el servidor Eureka y la caché del Gateway tardan un periodo de tiempo determinado en detectar la falta de latidos (heartbeats) y remover la instancia inactiva del registro.

Tarea







Verificación de saldos: Ana finalizó con $750.00 y Beto con $700.00.

Conclusión: La anotación @LoadBalanced en ClientesHttp.java del servicio transferencia logró que las peticiones internas de cargos y abonos para una misma operación fueran atendidas por réplicas distintas en orden Round Robin.